FuneralEase SA — Realistic Business Plan, Functional Spec & Technical Architecture

Funeral Policy Vault — Revised Edition (v3)

July 2026

Contents

1. Executive Summary **3**

2. Market Validation — What the Data Actually Supports **4**

3. Competitive Reality Check **5**
3.1 The Wedge 5

4. Problem Statement **6**

5. Business Model **7**
5.1 Revenue Projection (revised) 7
5.2 Go-To-Market — Organic & Self-Serve Only 7
5.3 What NOT to Build Into the Revenue Model 8

6. Functional Specification **9**
6.1 User Personas 9
6.2 Core Feature Set — Family Plan 9
6.3 Core Feature Set — Burial Society Module (self-serve creation) 9
6.4 Funeral Parlour Self-Service Portal 10
6.5 What This Product Deliberately Does Not Do 10

7. Non-Functional Requirements **11**

8. Technical Architecture **12**
8.1 High-Level Architecture 12
8.2 Subscription & Billing Architecture 12
8.3 Client Applications 13
8.4 Emergency Access Protocol 13
8.5 Security & Compliance 13
8.6 Offline Sync (Society Module) 13
8.7 Third-Party Integrations 14
8.8 Infrastructure 14

9. Technology Stack & Feature Scope **15**
9.1 Backend Stack Decision: C# / .NET 15
9.2 Admin Portal (Next.js) — Internal Staff Tool 15
9.3 API — Grouped by Domain 16
9.4 Native Apps (iOS Swift/UIKit MVVM, Android Kotlin MVVM) 16

10. Phased Roadmap **17**

11. Success Metrics (First 12 Months) **18**

12. Key Risks & Mitigations **19**

13. Overall Verdict **20**

Appendix A: Other Opportunities in the SA Death-Care & Bereavement Industry **21**

Appendix B: Brand Identity **22**

Appendix C: Two-Unit Architecture Deep Dive (Deferred) **26**

C.1 Domain Entities 26
C.2 Application Layer 28
C.3 Endpoints (Core API) 29
C.4 Infrastructure 30
C.5 What This Buys You, Concretely 32

1. Executive Summary

FuneralEase SA is a digital tool that helps South African households track every funeral policy they hold, get warned before a premium lapses, and know exactly what to do — and who to call — when a claim needs to be filed.

What changed in this revision, and why: the prior version's central claim — "no competitor solves this today" — is false. Grace Dignity Network already offers a free-forever "Life File" covering wills, insurance records, medical wishes, and funeral wishes, POPIA-compliant, with a paid tier and its own end-of-life services directory. This doesn't kill the idea, but it changes the strategy: FuneralEase SA cannot win by being "the cross-provider vault" in general, because that positioning is already taken, for free. It has to win by being *the specific tool that stops policies lapsing and claims getting declined* — a narrower, more operational claim that the data below says is where the real pain is, and that Grace Dignity Network's estate-planning framing doesn't appear to target directly.

This document folds the original business plan, functional specification, and technical architecture into one revised edition, corrected against real market research rather than assumption.

2. Market Validation — What the Data Actually Supports

Reliable numbers (consistent across sources):

- Roughly 28% of South African households hold at least one funeral policy — approximately **5 million households**, a proven, habitual-paying category.
- Funeral benefits are consistently the single most-complained-about product at the National Financial Ombud — **46.2% of Life Division cases closed in 2025**, with **declined claims the leading driver (56% in the most recent detailed breakdown)**. This is the strongest, most defensible data point in the plan and should anchor all messaging.
- Typical funeral cover premiums run R50–R500/month per policy, R200–R400/month for family cover — so a R9–29/month add-on is a modest incremental ask, not a new budget line.
- In funeral matters specifically, the Ombud's own reporting names the leading causes of complaint as **claims declined because terms and conditions weren't met, poor communication, insurable-interest disputes, waiting periods, and lapsing** — a near-exact match to the three failure modes this product targets.

Numbers to stop citing as settled:

- The prior version's "R4.4 billion annual funeral premium spend" figure (attributed to the FSCA) conflicts with other 2023 industry data putting funeral insurance premiums at **R15.2 billion** — roughly a 3.5x gap between sources. Don't lean on either figure specifically in external materials; the household-penetration number (28%, ~5M households) is better-sourced and does the same persuasive job.

Conclusion: the market-size and willingness-to-pay case is solid. The competitive case in the original plan was not, and has been rebuilt below around a narrower claim.

3. Competitive Reality Check

	Insurer apps (1Life, Ubuntu Life, Old Mutual, OUTsurance)	Grace Dignity Network	Funeral admin software (easi-Pol, WAPPoint)	FuneralEase SA
Built for	The insurer's own policyholders	The family, framed as estate/legacy planning	Funeral parlours' internal operations	The family, framed as insurance operations
Cross-provider view	No	Yes	No	Yes
Price	Bundled with cover	Free forever (Premium R49/mo for will/medical/financial)	B2B, R490+/mo	R9–29/mo
Per-policy premium tracking with lapse warnings	No	Not evidenced	N/A	**Yes — the core wedge**
Insurer-agnostic claims checklist	No	Not evidenced	N/A	**Yes — the second wedge**
Funeral crowdfunding	No	No	No	Deliberately not building — BackaBuddy already dominates (R157M+ raised in the past year, 0% platform fees, high trust)

Mandatory pre-build step: create a real Grace Dignity Network account and use its free tier end-to-end before Phase 1 begins. Confirm — don't assume — whether it already does per-policy premium tracking and lapse alerts. If it does, use the backup wedges in Section 12. If it doesn't, which the available evidence suggests, the wedge below holds.

3.1 The Wedge

Reframe the entire product around one sentence: **"Never let a funeral policy lapse without knowing, and never be caught without the right documents when you claim."**

- **Core value prop:** multi-policy premium tracker with escalating lapse warnings, targeting the *missed premiums lapsing cover unnoticed* failure mode.
- **Second value prop:** insurer-agnostic claims checklist, generated per policy the moment a claim is flagged in progress, targeting the *documentation not ready at claim time* failure mode.
- **Explicitly de-scoped from marketing (though still built, see Section 6):** general estate/legacy planning — that positioning belongs to Grace Dignity Network, already free. FuneralEase SA should never describe itself as "an estate planning tool."
- **Burial Society and Parlour Directory modules** stay in scope as self-serve retention/upsell features, not first-touch marketing hooks.

4. Problem Statement

South African households commonly hold multiple funeral policies — often stacked across insurers, employers, and informal community schemes — because no single policy is trusted to cover full costs. When a policyholder dies, beneficiaries frequently don't know which policies exist, where documents are kept, which insurer to call, or what the claims process requires, while under acute emotional stress and time pressure. Missed premium payments silently lapse cover with no family visibility until a claim is declined.

No funeral insurer has an incentive to help a family see policies held with a competitor — that structural gap is real. What has changed since the original plan is the recognition that an independent, non-insurer competitor (Grace Dignity Network) already occupies part of that gap, framed around estate planning rather than policy operations. FuneralEase SA's opportunity is the operational slice Grace Dignity Network does not appear to cover.

5. Business Model

The existence of a free-forever competitor changes the freemium math. A paid-only vault loses the comparison-shopping moment to Grace Dignity Network's free tier every time. The fix is to compete on the free tier too, and charge only for the operational layer that is actually differentiated.

Tier	Price	What's included
Free	R0	1 policy stored, manual premium reminders (in-app only), static claims checklist
Individual	R9/month	Unlimited policies, push + WhatsApp/SMS reminders with escalating lapse warnings, dynamic per-policy claims checklist
Family	R29/month	Everything in Individual, multiple household members, Emergency Access, full audit trail
Burial Society	R99–R249/month	Self-serve group workspace: contribution ledger, payout tracker, voting, dispute log
Parlour listing	Small monthly fee	Self-service directory listing — secondary revenue, not a growth target

5.1 Revenue Projection (revised)

Milestone	Assumption	MRR
Month 3	400 free signups, ~60 paid conversions (~15%)	~R1,200
Month 6	1,200 free, ~220 paid	~R4,500
Month 12	3,500 free, ~700 paid, first ~10 self-serve societies	~R14,000
Month 24	9,000 free, ~2,000 paid, ~50 societies, ~80 listings	~R45,000+

This is roughly a third of the original plan's projection at each milestone — the honest cost of a free tier existing to match a real competitor, rather than assuming an open field. Fund and staff the plan against these numbers, not the original ones.

5.2 Go-To-Market — Organic & Self-Serve Only

- **App Store Optimization (ASO)** as the primary channel: target high-intent search terms — "funeral policy tracker," "where do I find my mother's funeral policy," "funeral claim checklist South Africa," "burial society app" — where existing insurer apps don't rank for cross-provider intent.
- **Content marketing / SEO**: a small library of genuinely useful content ("What to do in the first 48 hours after a death in South Africa," "How to claim on a funeral policy — step by step," "Repatriating a body across provinces") funnelling into "start your family vault." Keep this anchored on the *lapse-prevention and claims-readiness* framing (Section 3.1), never general

"peace of mind" or "legacy" language, to avoid sounding like a weaker copy of Grace Dignity Network.

- **Referral loop built into onboarding**: inviting a family member as co-administrator or Emergency Contact is a required onboarding step, not optional — every household added this way is a warm referral.
- **Community content, not community partnerships**: share the same articles and short explainers in relevant Facebook groups and community WhatsApp broadcasts as a member sharing something useful, without needing formal sign-off.
- **What this deliberately avoids**: cold-calling burial societies, pitching NASASA, or funeral-parlour business development. Any of those relationships should be inbound — a user brings the app to their society, or a parlour finds the "claim your listing" page.

5.3 What NOT to Build Into the Revenue Model

- No automated insurer API integration — no open-insurance data standard exists in South Africa; manual entry and document upload is the correct architecture.
- No custody of funds, including for the Society module — route any contribution payment collection through an already-licensed payment service provider (Paystack, Stitch, or PayShap-enabled rails) as pure pass-through.

6. Functional Specification

6.1 User Personas

1. **Household Administrator** — sets up the vault, adds policies, designates beneficiaries and emergency contacts.
2. **Beneficiary / Family Member** — has emergency-access rights; normally read-only until an emergency access event is triggered.
3. **Society Creator/Treasurer** — self-serve creates and manages a burial-society workspace.
4. **Society Member** — joins a society workspace via invite link; views contributions and votes.
5. **Funeral Parlour Partner** — self-serve claims/manages their directory listing.

6.2 Core Feature Set — Family Plan

Policy Vault - Add a policy manually: insurer name, policy number, policy type, cover amount, premium amount, premium due date, beneficiary(ies), insurer claims contact/hotline. - Attach supporting documents (photo or PDF upload of the policy schedule, ID documents, proof of premium payment). - Optional OCR-assisted data entry: photograph the policy document, system pre-fills fields for user confirmation. **Treated as assistive only, not load-bearing** — real SA policy documents are frequently low-quality scans or hand-annotated, so the manual-entry path is the primary flow, not a fallback.

Premium Tracker & Reminders — the core wedge - Push notification + WhatsApp/SMS reminder ahead of each premium due date (configurable lead time). - "Lapsed cover" warning logic: escalating reminders if a due premium isn't marked paid within a grace window. - WhatsApp Business API bills per conversation — model this cost per subscriber explicitly before finalizing tier pricing (see Section 8.7).

Beneficiary Designation & Emergency Access - Household Administrator designates Emergency Contacts, choosing "immediate access" or "delayed access" (24–48 hour hold with notification to the Administrator, who can cancel). - Full audit trail of every access grant, view, and document download (see Section 8.4).

Claims Checklist — the second wedge - Step-by-step, insurer-agnostic checklist: death certificate (BI-1663/DHA-1663), notifying each insurer, required documents, submission timelines, expected payout timing. - Dynamically generated per-policy once a "claim in progress" flag is set.

Funeral Parlour Directory (self-serve population) - Location-based directory; parlours submit their own listing via a public "List your funeral home" form; basic verification (business registration check) before publishing. - In-app quote request/comparison tool for families.

Repatriation Guide & Grief Support Directory - Editorial, CMS-driven reference content — no partner integration required, kept current by the FuneralEase team.

6.3 Core Feature Set — Burial Society Module (self-serve creation)

- "Create a Society" flow: any user can start a society workspace in-app, become its default Treasurer, and invite members via a shareable link — no approval or onboarding from FuneralEase required.
- Member Registry, Contribution Ledger, Payout Tracker, Constitution Storage, Voting, Dispute Log — fully self-administered by the group that creates it.
- Upsell prompt to the paid Society tier once the group's free-tier usage limits are reached (member count or history length) — an in-product monetisation trigger, not a sales conversation.

6.4 Funeral Parlour Self-Service Portal

- Public “Claim your listing” web form: business name, registration number (self-reported, spot-checked), services, pricing bands, service area, photos.
- Lightweight moderation queue before a listing goes live.
- Once claimed, the parlour manages their own listing and receives quote requests directly — no account manager relationship required.

6.5 What This Product Deliberately Does Not Do

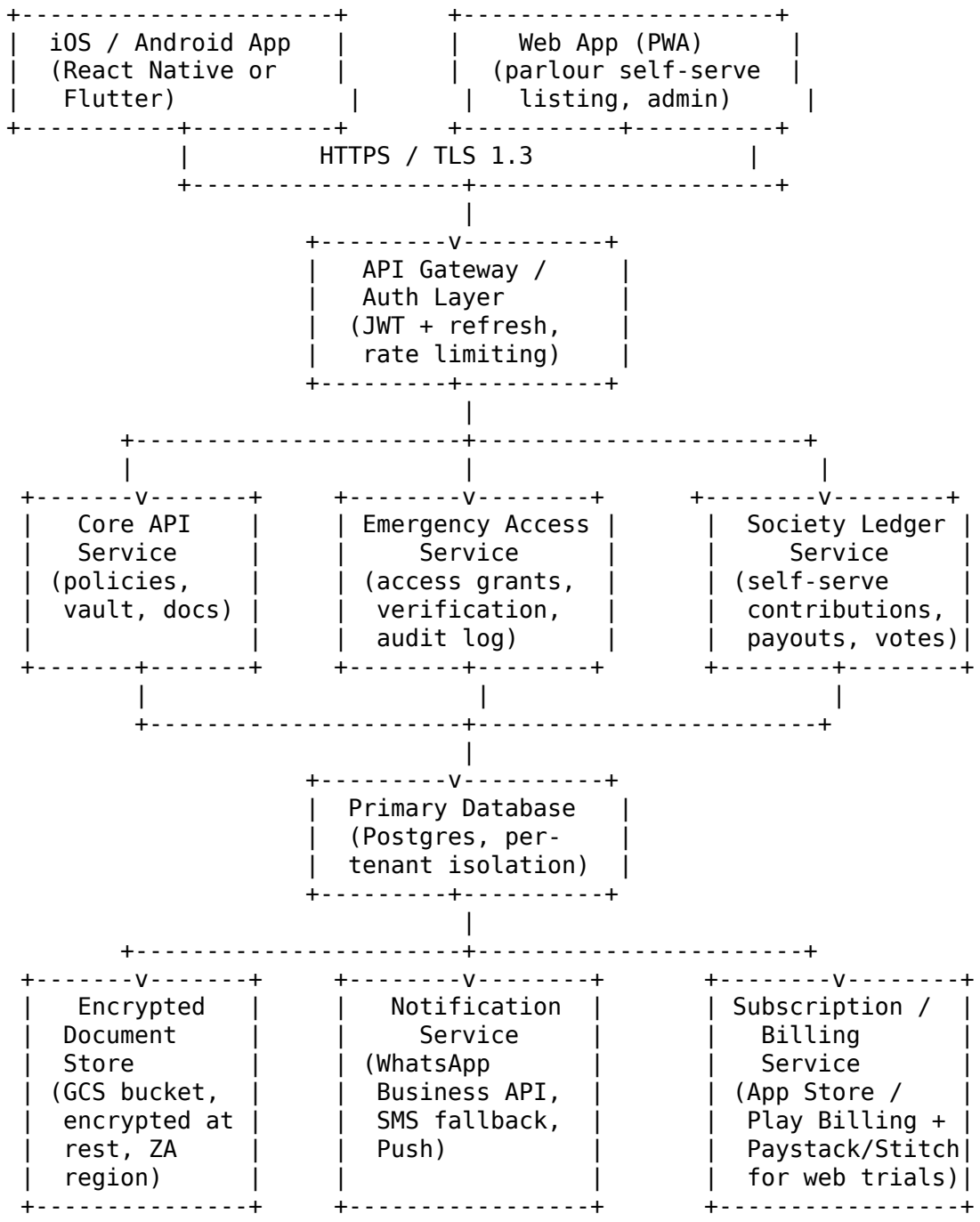
- General estate planning: wills, living wills, digital legacy, financial overview. This is Grace Dignity Network’s positioning, already free — competing there directly pushes users toward the free option instead of converting them.
- Funeral or memorial crowdfunding. BackaBuddy already dominates this specific adjacent need in South Africa.

7. Non-Functional Requirements

- **Availability:** target 99.5%+ uptime — this product is relied on during a crisis; the emergency-access flow is the highest-priority code path in the system.
- **Data protection:** POPIA compliance is foundational, not an add-on. Policy data includes ID numbers and financial information requiring explicit consent, a documented lawful basis, and retention handling. Engage a POPIA-qualified advisor before launch, not after.
- **Accessibility:** the interface must work well for lower digital-literacy users and lower-end Android devices, given a self-serve consumer funnel spans a wide range of technical comfort.
- **Time-to-value:** first meaningful value (one policy added, one emergency contact set) achievable within 5 minutes of install — critical for a trial-driven, self-serve funnel with no human onboarding.

8. Technical Architecture

8.1 High-Level Architecture



8.2 Subscription & Billing Architecture

Since growth depends entirely on self-serve conversion, billing infrastructure deserves the same design attention as the product itself:

- **Mobile subscriptions:** use native App Store / Google Play in-app subscription billing — non-negotiable on iOS, and it simplifies trial/renewal/cancellation logic considerably by leaning on platform-managed billing rather than building it in-house.

- **Web/trial entry point:** if a web-based trial or sign-up flow precedes app install, use a payment provider with strong South African support (Paystack or a Stitch-based flow) for web-initiated subscriptions, kept separate from mobile billing but reconciled against the same subscription-state record.
- **Single source of truth:** maintain one `subscription_status` record per household regardless of which billing channel it originated from, so a user who subscribes on iOS and later logs in on Android sees consistent access — a common self-serve SaaS pitfall worth getting right from day one.
- **Trial-to-paid conversion tracking:** instrument the funnel explicitly (install → first policy added → emergency contact set → trial reminder → conversion). In a self-serve model without a salesperson, this funnel data is the sales team.

8.3 Client Applications

- **Mobile:** cross-platform framework (React Native or Flutter) rather than native Swift/Kotlin — a single shared codebase reduces build and maintenance burden for a small team without a meaningful UX cost for this type of app.
- **Web:** lightweight React PWA for the parlour self-serve listing form and any admin/moderation tooling.

8.4 Emergency Access Protocol

The single most important design decision in the system:

1. **Pre-authorisation at setup:** Household Administrator names Emergency Contacts and chooses "immediate access" or "delayed access" (24–48 hour hold with notification, cancellable by the Administrator).
2. **Request triggers:** an Emergency Contact requests access via the app with their own registered phone/OTP confirmation — no death certificate required upfront, since that would defeat the purpose of fast access.
3. **Notification fan-out:** all other designated contacts and the Administrator (if reachable) are notified the moment a request is made.
4. **Full audit trail:** every access grant, view, and document download logged immutably with requester identity and timestamp.
5. **Time-boxed access:** grants a session (e.g. 30 days), not permanent control.

8.5 Security & Compliance

- TLS 1.3 in transit; AES-256 at rest; field-level encryption for ID numbers and other sensitive fields.
- South African (or contractually POPIA-compliant) cloud region for data residency.
- Explicit consent capture at signup; documented retention/deletion policy; a designated Information Officer role, as POPIA requires.
- Role-based access control enforced server-side for every persona (Administrator / Emergency Contact / Society Treasurer / Society Member / Parlour Partner).
- Automated encrypted backups with tested restore procedures.

8.6 Offline Sync (Society Module)

- Mobile client queues contribution entries locally when offline; syncs with server-assigned authoritative timestamps on reconnect.
- Conflict policy: "last confirmed write wins, prior version retained in history" — every change stays visible in the audit/dispute log regardless of which write "won," preserving transparency even without a human moderator (important in a self-serve model with no FuneralEase staff overseeing each group).

8.7 Third-Party Integrations

Integration	Purpose	Notes
App Store / Play Billing	Primary subscription billing	Required on iOS; simplifies renewal/cancellation logic
Paystack / Stitch (web)	Optional web-initiated trial/subscription	Reconcile against the same subscription-state record as mobile
WhatsApp Business API	Premium/contribution reminders	**Budget per-conversation cost explicitly against the R9/R29 tiers before finalizing pricing** — at scale this can meaningfully erode margin on the cheapest tier; SMS fallback
OCR (Google Vision / Document AI)	Optional pre-fill from photographed policy documents	Assistive only — user confirms every field, never treated as a dependency
Push notification service (FCM / APNs)	Reminders, emergency access alerts	Standard

8.8 Infrastructure

- Cloud provider with a South African or nearby compliant region (Google Cloud’s africa-south1, Johannesburg).
- Containerised services (Docker) behind a managed orchestration layer, sized modestly for a self-serve launch and scaled as subscriber count grows.
- CI/CD pipeline with staged environments and mandatory security review before production deploys.

9. Technology Stack & Feature Scope

9.1 Backend Stack Decision: C# / .NET

Three candidates were considered: Python (FastAPI), Kotlin (Spring Boot), and C# (.NET). C#/.NET is the recommendation, with a deliberately pragmatic — not maximalist — application of Domain-Driven Design.

Why not Python/FastAPI: it is the fastest path to the Phase 1 MVP and is genuinely the best fit for the OCR pre-fill work in Section 6.2. But the product's two hardest problems — the Emergency Access protocol (Section 8.4) and the premium lapse/escalation state machine (Section 6.2) — are exactly the kind of invariant-heavy, correctness-critical logic where a dynamically-typed, loosely-structured framework tends to accumulate quiet bugs as a team grows. Building the system's single most important design decision (Section 8.4) on the most permissive stack is an avoidable risk.

Why not Kotlin/Spring Boot: a genuinely strong alternative, and the right call if the team wants one language shared across backend and Android. Not a wrong answer for this product — a legitimate trade worth revisiting if hiring or team composition changes.

Why C#/.NET, concretely:

- The domain is insurance-operations-shaped (policies, premiums, lapse states, claims), and .NET's ecosystem (EF Core, ASP.NET Identity, policy-based authorization) is proven in exactly that kind of compliance-heavy, financial-adjacent line-of-business software, including at South African insurers and banks.
- POPIA's specific requirements already scoped in Section 8.5 (field-level encryption, RBAC per persona, immutable audit trail) map onto mature, built-in .NET primitives rather than custom plumbing.
- Local hiring depth: .NET has deeper enterprise penetration in South Africa than FastAPI specifically, which matters if the team ever needs to bring in a contractor.

Scope DDD tactically, not everywhere. Full textbook DDD (CQRS, event sourcing, aggregates for every entity) would blow the Phase 1 timeline. Apply aggregate roots, value objects, and domain events only to the two areas with real invariants — the **Policy/Premium lifecycle** and the **Emergency Access session** — plus the **Society Ledger** once that module is built (Section 6.3). Everything else (parlour listings, CMS content, notification templates) stays plain CRUD. That distinction is what keeps DDD a tool rather than ceremony.

9.2 Admin Portal (Next.js) — Internal Staff Tool

- Staff login with 2FA; roles: support agent, content editor, super admin
- Dashboard: MRR by tier, trial-to-paid conversion, active households, churn, referral coefficient (Section 11 metrics)
- Household lookup: search by phone/email, view subscription tier and policy count (not contents), manual overrides (extend trial, apply credit) — all writes go through the audit log
- Content/CMS editor: claims checklist templates, repatriation guide articles, grief support directory, SEO content library
- Parlour listing moderation queue: approve/reject self-serve submissions, business-registration spot checks, handle reported listings
- Society moderation: view reported societies and escalated disputes, suspend a workspace if abused
- Billing reconciliation view: cross-check App Store / Play / Paystack-Stitch against the single `subscription_status` source of truth; surface failed payments
- WhatsApp/SMS template manager: approval status, cost-per-conversation monitoring, test sends
- Immutable audit log viewer: emergency access grants/views, staff actions, POPIA-relevant data access

- Feature flags: roll Society/Parlour modules out gradually by app version
- CSV/report export for investor updates

9.3 API — Grouped by Domain

- **Auth:** signup/login (phone OTP), JWT + refresh, multi-user household membership, separate staff auth scope, consent capture
- **Policy vault:** CRUD, document upload via presigned URL to encrypted store, OCR pre-fill (assistive only)
- **Premium tracker:** due-date schedule engine, reminder dispatch job, escalating lapse-warning logic, mark-as-paid
- **Claims:** flag claim in progress, generate dynamic per-policy checklist, status tracking
- **Emergency Access:** contact designation, OTP-verified access request, notification fan-out, time-boxed session issuance, immutable audit write, admin revoke
- **Society:** workspace creation, invites, contribution ledger, payout tracker, voting, dispute log, offline-sync conflict resolution
- **Parlour directory:** self-service listing submission, public search, quote request relay
- **Billing:** App Store/Play server-notification webhooks, Paystack/Stitch webhooks, entitlement checks
- **Notifications:** WhatsApp Business API, SMS fallback, push dispatch (FCM/APNs)
- **Content:** serves CMS content to apps and web
- **Compliance:** POPIA subject-access export, deletion/retention jobs
- **Analytics ingestion:** funnel events (install → first policy → emergency contact → conversion)
- **Admin-only endpoints:** everything the portal needs, scoped and authorized separately from the consumer API

9.4 Native Apps (iOS Swift/UIKit MVVM, Android Kotlin MVVM)

- Onboarding: free trial (no card), first policy + first emergency contact inside 5 minutes
- Policy vault: list/detail, add/edit (manual entry + camera-assisted OCR capture), document viewer
- Premium tracker: due-date view, mark-as-paid, push handling, lapse-warning banners
- Claims: start-a-claim flow, dynamic checklist, document ticking, status
- Emergency Access (the family-member side): OTP request flow, immediate/delayed access UX, time-boxed countdown banner
- Household management: invite co-administrators, beneficiary designation, role indicator
- Burial Society: create/join flow, ledger view, payout tracker, voting, dispute log, offline queue indicator
- Parlour directory: browse, view listing, request/compare quotes
- Repatriation guide and grief directory content browser
- Subscription management: tier view, trial countdown, native store subscription deep link
- Settings: notification channel preferences, POPIA consent and data export/delete, biometric app lock, given the data sensitivity
- Referral: invite-a-family-member step in onboarding, post-resolution share prompt
- Accessibility: large-text support, tested on lower-end Android hardware

10. Phased Roadmap

Phase 0 (Week 0 — new): Competitive Audit Create a real Grace Dignity Network account, use its free tier end-to-end, and confirm in writing whether it already does per-policy premium tracking with lapse alerts. This gate must be cleared before Phase 1 begins — if the audit turns up a closer overlap than expected, pivot to the backup wedges in Section 12 before any further build spend.

Phase 1 (Weeks 1–8): Family Vault MVP + Billing Policy vault, premium reminders, emergency access, claims checklist, App Store/Play subscription billing, trial funnel instrumentation, free tier live from day one. Goal: a fully self-serve, install-to-paid-subscriber funnel working end to end.

Phase 2 (Weeks 9–14): Content & ASO Engine Publish the core content library optimised for the high-intent search terms in Section 5.2. Launch referral mechanics (invite-a-family-member flow, post-resolution review/share prompt).

Phase 3 (Weeks 15–20): Self-Serve Society & Parlour Modules "Create a Society" flow and self-serve parlour listing form — both designed to require zero manual onboarding, with lightweight moderation only.

Phase 4 (Ongoing): Compounding Growth Expand the content library based on actual user search-query data; iterate referral incentives; consider paid App Store Search Ads only once organic funnel economics (trial-to-paid conversion, referral coefficient) are well understood.

11. Success Metrics (First 12 Months)

- Trial-to-paid (and free-to-paid) conversion rate — the single most important number in this model.
- Time-to-first-value (install → first policy + emergency contact added).
- Organic install share vs. paid (goal: organic dominant by month 6).
- Referral coefficient (new households added per existing household via invite/share).
- Self-serve societies and parlour listings created (proof the self-serve model works without manual involvement).
- Monthly recurring revenue by plan tier, tracked against the revised projections in Section 5.1, not the original plan's.

12. Key Risks & Mitigations

Risk	Mitigation
Grace Dignity Network’s free tier already does the core job	Complete the Phase 0 audit before any build spend; fall back to a B2B white-label play (parlours/burial societies as the customer, not individual consumers) or make the Burial Society module the primary product instead of the policy vault
Free tier cannibalizes paid conversion more than modeled	Instrument free-to-paid conversion from day one; treat Section 5.1’s projections as a hypothesis, not a promise
WhatsApp reminder costs erode thin per-user margin	Model cost-per-subscriber before finalizing pricing; consider push-only reminders on the cheapest tier
Marketing drifts into “estate planning” framing and loses to the free competitor on its own ground	Keep all copy anchored on the Ombud statistics and lapse/claims-declined framing; never “peace of mind” or “legacy planning” language
Self-serve Society/Parlour features attract fraudulent or low-quality listings without vetting	Lightweight automated + spot-check moderation; user-reporting mechanism; business-registration spot checks for parlour listings
Slower early growth without partnership-driven distribution	Front-load content/SEO investment in Phase 2 so the organic engine is compounding before paid acquisition would otherwise be needed

13. Overall Verdict

Proceed, conditionally — but only after the Phase 0 competitive audit, and against the narrower wedge, revised pricing, and reduced revenue expectations in this document.

What's still genuinely strong: - A real, quantified, regulator-documented pain point: funeral benefits are the most-complained-about product at the National Financial Ombud, and declined claims — commonly from lapsed premiums and missing documentation — are the leading cause. - A proven, habitual-spending base: roughly 5 million South African households already pay R50–500/month for funeral cover, making a R9–29/month add-on a modest incremental ask. - Low technical and regulatory complexity relative to most other consumer fintech ideas: no payment custody, no debit-order licensing, no dependency on insurer cooperation.

What's genuinely uncertain: - This is a prevention product addressing a latent risk, not a felt, immediate one — a harder conversion sell than funeral cover itself. - A real, free, well-positioned competitor (Grace Dignity Network) already exists and was missed in the original plan — the wedge in this revision assumes a feature gap that must be verified first-hand, not assumed. - Real trial-to-paid and referral-coefficient numbers are unknown until launch; the first 90 days of live data will be more informative than any further desk analysis.

Recommendation: complete the Phase 0 audit first. If the wedge holds, proceed with the narrowed scope, revised pricing, and conservative revenue targets in this document. Anchor all marketing on the Ombud complaint statistics rather than generic "peace of mind" framing, and be ready to iterate hard on messaging and positioning before concluding anything further about the product itself.

Appendix A: Other Opportunities in the SA Death-Care & Bereavement Industry

Surveyed while researching this plan — not recommended for immediate pursuit, but worth knowing about:

- **Deceased estate document-prep tool** (helping next-of-kin assemble the correct paperwork before reporting to the Master of the High Court) — the most promising unexplored gap found in this research. Government has launched its own *Deceased Estate Online Registration System*, but that is back-end infrastructure for the Master's office, not a citizen-facing tool that tells an ordinary person which documents they need before they queue. A significant share of the well-documented Master's Office delays stem from incomplete or incorrect submissions — a guided checklist/document-prep product (not a legal filing service) could genuinely reduce that friction, and no direct consumer-facing competitor was found in this research.
- **Funeral parlour price transparency/comparison** — funeral costs vary from R15,000 to over R100,000 with little standardised price comparison available to grieving families making fast decisions. A "PriceCheck for funerals" is a plausible gap; a prior attempt (Sendoff, a full-service digital funeral arranger launched in 2021) exists but its current status is unclear and would need direct investigation before assuming the lane is open.
- **Funeral/memorial crowdfunding — avoid.** BackaBuddy already dominates this in South Africa (R157 million+ raised in the past 12 months, 0% platform fees since mid-2025, high trust, 650,000+ donors). Not a viable wedge.
- **Customary-law intestate succession guidance** (for informal/customary marriages, multiple spouses, extended-family disputes) — a real and complex pain point, but overlaps significantly with existing legal-aid content and Grace Dignity Network's estate guidance; would need a sharper angle to be more than a content play.
- **Grief counseling access/telehealth marketplace** — no SA-specific bereavement-focused telehealth marketplace was found, but monetization is difficult: the target user has often just lost a household's primary earner and has low ability to pay for counseling out of pocket.

This document is a planning artifact, not a legal, financial, or compliance opinion. Confirm all competitive claims first-hand, and engage a POPIA-qualified advisor before launch.

Appendix B: Brand Identity

A first pass at a visual identity for FuneralEase SA — palette, typography, logo, and the theme applied to the policy vault screen described in Section 6.

The signature mark is an arch — a doorway, not a headstone — with a small flame resting inside that doubles as a keyhole: remembrance and access to the vault in one shape. Dusk ink stands in for funeral black or clinical white; sage marks a policy that’s paid and in order; candlewick gold flags a premium due soon; ember is held back for the one moment — overdue — where the palette needs to raise its voice.

BRAND IDENTITY

Never let a policy lapse without knowing.

FuneralEase SA is the quiet, operational layer behind a family's funeral cover — every policy in one place, every premium tracked, every claim ready before it's needed.



PALETTE

Five colours, each with one job

Dusk ink
#1C2431

Warm paper
#F3EEE3

Candlewick
#E3A54F

Sage continuity
#5E8571

Ember
#A6462D

TYPOGRAPHY

Three roles, never mixed

LORA — THE FAMILY-FACING VOICE, HEADLINES ONLY

Every policy, accounted for.

IBM PLEX SANS — INTERFACE AND BODY TEXT

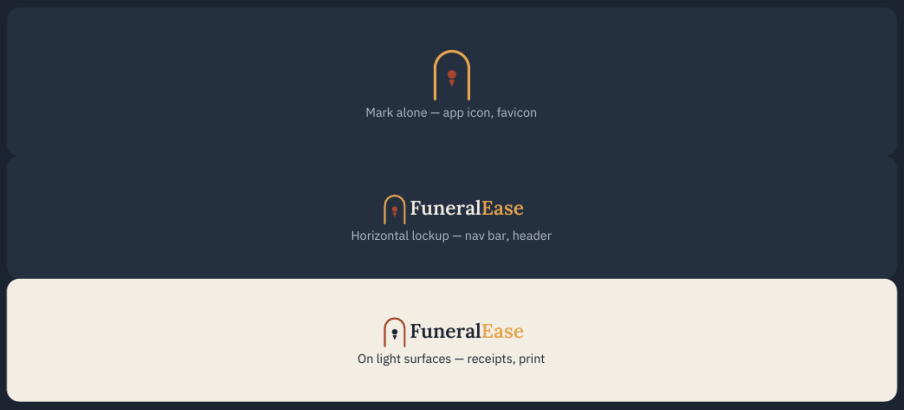
Add a policy, set a reminder, and know exactly what to do when it's time to claim. No jargon, no fine print buried in a drawer.

IBM PLEX MONO — POLICY NUMBERS, AMOUNTS, DATES

`Policy #KH-22910 · R249/month · Due 3 Aug`

LOGO USAGE

Mark and lockup

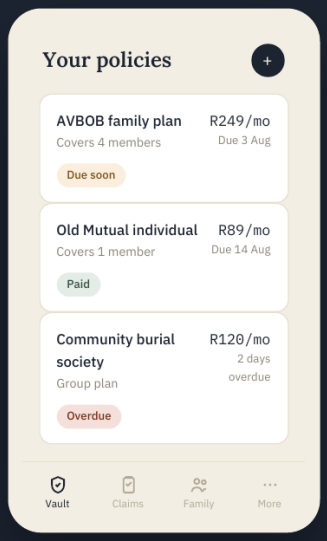


APPLIED

The theme, in the product

Sage means paid and in order. Candlewick means a premium is due soon. Ember is reserved only for what's overdue — the one moment the palette raises its voice.

Mono-spaced amounts and dates sit inside plain sans cards, so the numbers a family actually needs read like data, not decoration.



FuneralEase SA — brand presentation, v1

Dusk ink · Warm paper · Candlewick · Sage · Ember

ewpage

Appendix C: Two-Unit Architecture Deep Dive (Deferred)

This is the detailed design for the deferred “split worker into its own deployable unit + queue” option — not for Phase 1, but fully specified so it’s ready the moment the trigger (Section 9, Pass 4) is hit: nightly jobs regularly exceeding a few minutes, or background load measurably affecting API p95 latency.

C.1 Domain Entities

Six aggregate roots. Each owns its invariants; nothing outside the aggregate mutates its internals directly.

```
public class Household : AggregateRoot
{
    public HouseholdId Id { get; }
    public List<HouseholdMember> Members { get; } = new();
    public List<PolicyId> PolicyIds { get; } = new();

    public void AddMember(HouseholdMember member, HouseholdMember actingAs)
    {
        if (actingAs.Role != HouseholdRole.Administrator)
            throw new DomainException("Only an Administrator can add household members.");
        Members.Add(member);
        Raise(new HouseholdMemberAdded(Id, member.Id));
    }
}

public class Policy : AggregateRoot
{
    public PolicyId Id { get; }
    public HouseholdId HouseholdId { get; }
    public string InsurerName { get; private set; }
    public Money PremiumAmount { get; private set; }
    public DueDate NextDueDate { get; private set; }
    public LapseStatus Status { get; private set; } = LapseStatus.Current;
    public List<Beneficiary> Beneficiaries { get; } = new();

    public void MarkPremiumPaid(DateOnly paidOn)
    {
        if (Status == LapseStatus.Lapsed)
            throw new DomainException(
                "A lapsed policy must be manually reinstated, not marked paid.");
        NextDueDate = NextDueDate.Advance();
        Status = LapseStatus.Current;
        Raise(new PremiumMarkedPaid(Id, HouseholdId, paidOn));
    }

    public void Escalate(LapseStatus newStatus)
    {
        if (newStatus <= Status) return;
        Status = newStatus;
        Raise(new PremiumLapseEscalated(Id, HouseholdId, newStatus));
    }
}

public enum LapseStatus { Current, DueSoon, Overdue, Lapsed }

public class EmergencyAccessSession : AggregateRoot
```

```
{
    public HouseholdId HouseholdId { get; }
    public List<EmergencyContact> Contacts { get; } = new();
    public List<AccessGrant> Grants { get; } = new();
    private readonly List<AuditEntry> _audit = new();
    public IReadOnlyList<AuditEntry> Audit => _audit.AsReadOnly();

    public AccessGrant RequestAccess(ContactId requesterId, bool otpVerified)
    {
        if (!otpVerified)
            throw new DomainException("Emergency access requires OTP verification.");
        var contact = Contacts.SingleOrDefault(c => c.Id == requesterId)
            ?? throw new DomainException("Requester is not a designated emergency contact.");
        var grant = AccessGrant.Create(
            contact,
            delayHours: contact.AccessMode == AccessMode.Delayed ? 36 : 0);
        Grants.Add(grant);
        _audit.Add(AuditEntry.For(grant, AuditAction.Requested));
        Raise(new EmergencyAccessRequested(HouseholdId, requesterId, grant.Id, grant.ActivatesAt));
        return grant;
    }
}

public class Claim : AggregateRoot
{
    public ClaimId Id { get; }
    public PolicyId PolicyId { get; }
    public List<ChecklistItem> Checklist { get; } = new();
    public ClaimStatus Status { get; private set; } = ClaimStatus.InProgress;

    public void Complete()
    {
        if (Checklist.Any(i => i.Required && !i.Checked))
            throw new DomainException("Cannot complete a claim with unchecked required items.");
        Status = ClaimStatus.Submitted;
        Raise(new ClaimSubmitted(Id, PolicyId));
    }
}

public class Subscription : AggregateRoot
{
    public HouseholdId HouseholdId { get; }
    public SubscriptionTier Tier { get; private set; }
    public BillingChannel Channel { get; private set; }
    public DateTimeOffset EntitlementExpiresAt { get; private set; }

    // Never mutated directly by a webhook handler — see BillingEventLog, Section 4.
    public void ApplyEvent(BillingProviderEvent evt)
    {
        Tier = evt.ResolvedTier;
        Channel = evt.Channel;
        EntitlementExpiresAt = evt.ExpiresAt;
        Raise(new SubscriptionStateChanged(HouseholdId, Tier, EntitlementExpiresAt));
    }
}

// Deferred module — kept here for shape continuity, not built in Phase 1
public class SocietyWorkspace : AggregateRoot
{
    public List<LedgerEntry> Ledger { get; } = new();
```

```
    public int Version { get; private set; } // optimistic concurrency token
}
```

Confidence: High. These shapes don't change whether there's one deployable unit or two — the aggregate boundaries are the same either way. This is the part worth locking in regardless of the infrastructure question.

C.2 Application Layer

One command or query per use case, thin handlers, domain does the real work. The split that matters here is **which handlers run in Core API (synchronous, user-facing) versus Worker (asynchronous, side-effect-heavy)**.

```
// Runs in Core API — must return fast
public record MarkPremiumPaidCommand(PolicyId PolicyId, DateOnly PaidOn) : IRequest<Result>;

public class MarkPremiumPaidHandler : IRequestHandler<MarkPremiumPaidCommand, Result>
{
    private readonly IPolicyRepository _policies;
    // wraps EF Core SaveChanges + outbox insert, one transaction
    private readonly IUnitOfWork _uow;

    public async Task<Result> Handle(MarkPremiumPaidCommand cmd, CancellationToken ct)
    {
        var policy = await _policies.GetAsync(cmd.PolicyId, ct);
        policy.MarkPremiumPaid(cmd.PaidOn);
        // persists Policy + writes PremiumMarkedPaid to Outbox, one transaction
        await _uow.CommitAsync(ct);
        return Result.Success();
    }
}

// Runs in Core API — the OTP-gated, latency-critical path
public record RequestEmergencyAccessCommand(
    HouseholdId HouseholdId,
    ContactId RequesterId,
    string OtpCode) : IRequest<AccessGrantDto>;

// Runs in Worker — triggered by an event from the outbox/queue, not by an HTTP request
public class SendPremiumReminderOnDueSoon : INotificationHandler<PremiumLapseEscalated>
{
    private readonly IWhatsAppSender _whatsApp;
    private readonly ISmsSender _smsFallback;

    public async Task Handle(PremiumLapseEscalated evt, CancellationToken ct)
    {
        try
        {
            await _whatsApp.SendReminderAsync(evt.HouseholdId, evt.NewStatus, ct);
        }
        catch (WhatsAppDeliveryException)
        {
            await _smsFallback.SendReminderAsync(evt.HouseholdId, evt.NewStatus, ct);
        }
    }
}

// Runs in Worker — billing webhooks are received by Core API but *processed* here
public class ApplyBillingEventHandler : INotificationHandler<BillingEventReceived>
```

```
{
    private readonly ISubscriptionRepository _subs;
    public async Task Handle(BillingEventReceived evt, CancellationToken ct)
    {
        var subscription = await _subs.GetAsync(evt.HouseholdId, ct);
        // idempotent: BillingEventLog already deduped, see Section 3
        subscription.ApplyEvent(evt.ToDomainEvent());
        await _subs.SaveAsync(subscription, ct);
    }
}
```

Which handlers go where:

Handler	Runs in	Why
Auth, policy CRUD, mark-paid, claims checklist reads/writes	Core API	User is waiting on the response
`RequestEmergencyAccessCommand` (the OTP-verified grant itself)	Core API	Must be synchronous and fast — this is the path Section 7 calls highest-priority
Emergency access **notification fan-out** to other contacts	Worker	The grant already succeeded; notifying others is a side effect, not the user's wait
Reminder dispatch, lapse escalation scan	Worker	Scheduled, bulk, exactly the load that shouldn't share a process with Emergency Access
Billing webhook **receipt** (return 200 fast)	Core API	Providers expect a fast ack or they retry aggressively
Billing webhook **processing** (apply to Subscription)	Worker	Can tolerate seconds of delay; keeps webhook spikes off the API
OCR pre-fill	Worker	Already async/assistive per Section 6.2 — user isn't blocked on it

Confidence: High on the split logic itself. **Medium** on drawing the Emergency Access line exactly where I did — reasonable people could argue notification fan-out is important enough to also be synchronous; I'd keep it async unless real usage shows contacts need instant fan-out confirmation.

C.3 Endpoints (Core API)

```
POST   /auth/otp/request
POST   /auth/otp/verify
POST   /auth/refresh

GET    /households/{id}
POST   /households/{id}/members
GET    /policies?householdId=
POST   /policies
PATCH  /policies/{id}
POST   /policies/{id}/mark-paid
POST   /policies/{id}/documents          (presigned upload URL)
```

```
POST   /claims                              (start a claim, links to a policy)
GET    /claims/{id}/checklist
PATCH  /claims/{id}/checklist/{itemId}
POST   /claims/{id}/complete

POST   /emergency-access/contacts
POST   /emergency-access/request            (OTP-gated, returns grant)
GET    /emergency-access/grants/{id}
POST   /emergency-access/grants/{id}/revoke

GET    /subscriptions/{householdId}
POST   /webhooks/appstore                   (ack fast, log + outbox, 200)
POST   /webhooks/playstore
POST   /webhooks/paystack

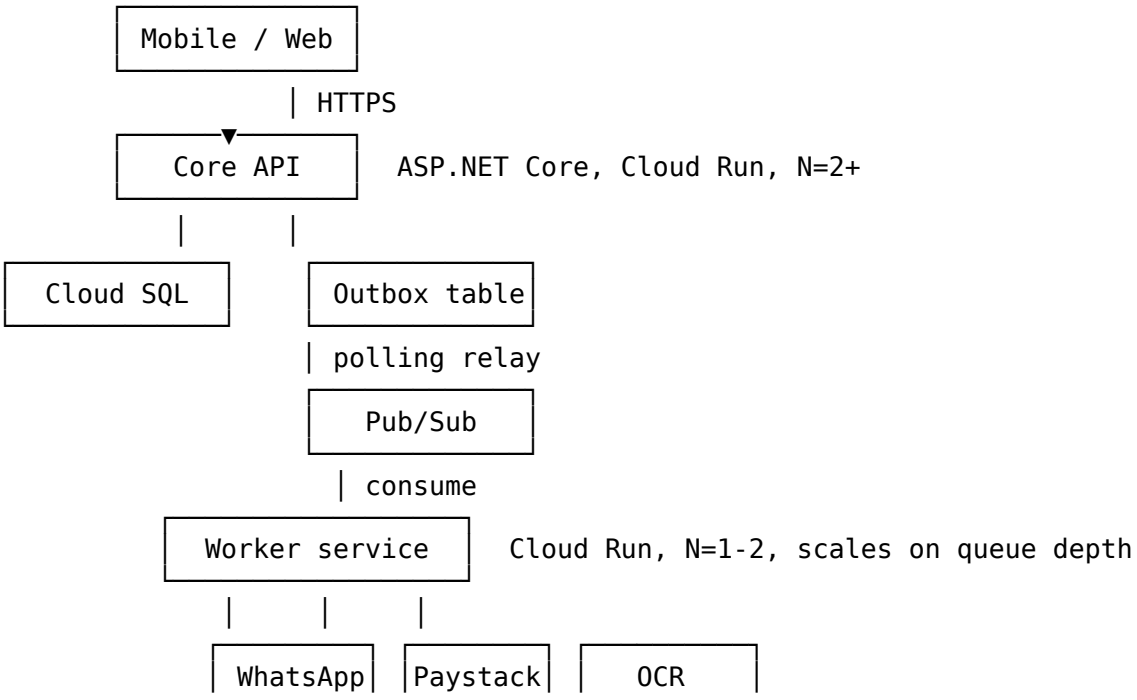
GET    /parlours?area=
POST   /parlours                             (self-service listing submission)

GET    /content/claims-checklist-templates
GET    /content/repatriation-guide

# Admin-only, separate auth scope
GET    /admin/households/search
GET    /admin/metrics/dashboard
GET    /admin/audit-log
POST   /admin/parlours/{id}/moderate
```

Confidence: High. This is a straightforward REST surface; the interesting decisions already happened in the application layer above, not in the route table.

C.4 Infrastructure





Cloud SQL runs PostgreSQL in the africa-south1 (Johannesburg) region — the Outbox table lives in this same instance, written in the same transaction as the aggregate change (Section C.4 below explains why that matters more than the queue choice).

Core API is the only component that touches Emergency Access, and it has no dependency on the Outbox, SQS, or Worker to serve that request — those exist purely to carry side effects (reminders, billing processing, OCR) off the request path.

Key infrastructure decisions:

Decision	Detail	Confidence
Outbox lives in the same Cloud SQL instance as the aggregates, written in the same transaction	This is what actually solves the reliability problem — not the queue. If the outbox write and the aggregate write aren't atomic, you're back to the dual-write problem this whole design exists to avoid.	**High**
Google Cloud Pub/Sub as the queue, not RabbitMQ/Kafka self-hosted	At this scale, a managed queue with no servers to patch is strictly less operational burden than running your own broker. Kafka in particular would be significant overkill for this message volume.	**High**
Relay from outbox → Pub/Sub is a small polling task, not a new heavyweight service	It has one job: read unpublished outbox rows, publish to Pub/Sub, mark published. Doesn't need its own deployable unit — can run as a Cloud Run job on a schedule, or a hosted background task inside Core API.	**Medium-High**
Worker scales independently on Pub/Sub subscription backlog	This is the actual payoff of the split — reminder bursts around month-end due dates scale the Worker's Cloud Run instances without touching Core API capacity at all.	**High**
`BillingEventLog` table, keyed on `(provider, event_id)`, checked before any Subscription mutation	Makes webhook processing idempotent regardless of provider retries or out-of-order delivery — this was flagged as a correctness gap in the critique, not a scale gap, and holds whether there's one deployable unit or two.	**High**

Decision	Detail	Confidence
Both Core API and Worker connect to the same Cloud SQL primary (no replica yet)	Consistent with the Section 9 final recommendation — a replica is a *separate*, later decision gated on its own trigger (admin/reporting queries contending with live traffic), not bundled into this split.	**Medium**
Worker is a single Cloud Run service, not split further by job type	Splitting reminders / billing / OCR into separate services each would be over-engineering at ~700 households. One Worker, multiple hosted background jobs inside it, is enough until a specific job type becomes the bottleneck.	**Medium-High**

C.5 What This Buys You, Concretely

Once built, a burst of premium reminders at month-end, a slow OCR provider, or a flood of billing webhooks can no longer add latency to `POST /emergency-access/request` — that endpoint's only dependency is Core API and Postgres, full stop. That's the entire justification for the added operational surface, and it's worth naming explicitly: **build this when you have evidence Emergency Access latency is coupled to background load, not on a calendar date.**